

Evaluating Open-Source LLMs for Software Security and Functional Validation: A Comparative Analysis

Group 7 : Mohemad Kudia(2110974), Yasteel Ramphal(2108498), Ziphokazizamagcugcwa Nzama(1830373)

School of Electrical and Information Engineering, University of Witwatersrand, Johannesburg, 2000, South Africa

Abstract---This paper examines the effectiveness of open-source large language models (LLMs) in analysing code security, particularly their ability to detect vulnerabilities and validate functionality. We tested five open-source LLMs from the HuggingFace leaderboard using the OWASP Benchmark for Java. Our approach compared zero-shot and chain-of-thought prompting across three tasks: vulnerability detection, functional validation, and a combined assessment. The results show that while these models can identify common security flaws, they often miss subtle vulnerabilities and fail to recognize cross-module dependencies. Chain-of-thought prompting consistently enhanced performance, especially for complex vulnerabilities requiring multi-step reasoning. Our findings indicate that open-source LLMs can support initial code reviews when used with effective prompting strategies, but they are not yet a substitute for specialized security tools or expert analysis.

Keywords: Large language models, open-source AI, security vulnerability detection, code analysis, OWASP Benchmark, chain-of-thought prompting, functional validation, software security, static analysis

I. Introduction

A. Background and Context

Large Language Models (LLMs) have become powerful tools in software development, changing how developers write, review, and debug code. Recent improvements show that these models can understand programming languages, generate working code, and even detect potential software flaws [1]. As LLMs are increasingly used in software engineering, researchers and practitioners are exploring their role in critical tasks like code review—a process that typically requires deep expertise and considerable time.

Code review is essential for secure software development, acting as a key quality check to catch bugs, improve maintainability, and strengthen security before deployment. However, manual reviews are labour-intensive, requiring skilled developers to analyse code line by line [2]. This approach is not only slow but also prone to human error, especially under tight deadlines or fatigue. Mistakes in code reviews can have serious consequences, potentially introducing security weaknesses that lead to data breaches, financial harm, or risks to user safety.

Proprietary LLMs, such as GitHub Copilot, are widely used for code generation and analysis. However, open-source alternatives offer important benefits, including transparency, lower costs, and customization, making them accessible to more developers and organizations [3]. Despite these advantages, little research has examined how well open-source LLMs perform in critical tasks like security vulnerability detection and functional validation.

B. Problem Statement

Despite the potential benefits of open-source LLMs in software development, their capabilities in security analysis and functional validation remain understudied. Pearce et al. [4] investigated zero-shot vulnerability repair using language models but primarily focused on proprietary

systems rather than open-source alternatives. This research gap raises critical questions about whether open-source LLMs can effectively:

1. **Flag code security vulnerabilities** with accuracy comparable to proprietary models.
2. **Perform software functional validation** to ensure code behaves as intended.
3. **Address both security and functional correctness** simultaneously.
4. **Provide meaningful, actionable feedback** on detected vulnerabilities.

These questions are particularly important because functional correctness and security are closely linked. Many security vulnerabilities arise from logical errors, faulty implementations, or deviations from expected behaviour. When code fails to function as designed, it can introduce risks such as flawed input validation, unauthorized access, or data leaks. As a result, thorough functional validation is a key part of identifying and mitigating security vulnerabilities.

C. Research Objectives

This study aims to systematically evaluate the capabilities of open-source LLMs in enhancing code review processes, with the objectives being to:

1. Assess the effectiveness of at least five open-source LLMs from the Hugging Face Open LLM Leaderboard in identifying common security vulnerabilities in code.
2. Evaluate these model's ability to validate software functionality and detect functional errors that could lead to security issues.
3. Compare the performance of zero-shot prompting versus chain-of-thought prompting techniques across both security and functional validation tasks.
4. Analyze the quality, accuracy, and usefulness of vulnerability descriptions generated by these models.
5. Identify potential integration strategies for incorporating open-source LLMs into existing development workflows.

II. Related Work

A. Code Review in Software Development

Code review practices have undergone significant changes, shifting from formal inspections to more informal, tool-assisted approaches. Sadowski et al. [5] conducted an extensive study of modern code review practices at Google, finding that while developers valued reviews for maintaining code quality, the main benefits were knowledge sharing and improved team awareness. Their research showed reviewers typically spent 15-30 minutes per code change, focusing more on understanding the code's purpose and design rather than exhaustive error checking. This time pressure underscores the need for automated support in code reviews, especially for detecting security vulnerabilities that require specialized expertise or careful analysis.

McIntosh et al. [6] examined how code review coverage and participation affect software quality across multiple open-source projects. Their work revealed that code changes receiving less thorough reviews tended to have more defects after release, highlighting the critical role

of code review. They also found that both the amount of code examined, and reviewer engagement significantly predicted software quality. However, their study confirmed that achieving complete review coverage remains difficult, as resource limitations often restrict how deeply and broadly code can be examined.

The pull-based development model, now widely adopted in both open-source and commercial projects, has transformed code review practices. Gousios et al. [7] analysed this model across 291 GitHub projects, studying how pull requests support distributed development and code review. Their research showed that while pull requests enable more transparent and collaborative reviews, project maintainers often face challenges managing high volumes of contributions and communicating effectively with contributors. These bottlenecks can result in delayed feedback and potentially superficial evaluations of code quality and security.

Together, these studies reveal several persistent challenges in modern code review:

1. Limited review time restricts how thoroughly code can be examined.
2. Detecting certain defects, especially security vulnerabilities, requires specialized expertise.
3. Growing numbers of contributions in collaborative projects overwhelm review capacity.
4. Maintaining review quality while keeping pace with rapid development creates ongoing tension.

These challenges demonstrate the increasing importance of automated support for code review. While traditional static analysis tools address some needs, they generate many false positives and need expert configuration. LLMs present a more adaptable alternative, capable of understanding code meaning and spotting issues that pattern-matching tools might miss. However, whether open-source LLMs can effectively support security-focused code reviews remains uncertain.

B. Large Language Models in Software Engineering

Recent years have witnessed a remarkable transition in software engineering as Large Language Models (LLMs) have moved from research prototypes to practical development aids. A comprehensive study by Xu et al. [8] systematically assessed code-capable LLMs across multiple programming languages and problem domains. Their analysis revealed an important finding: performance fluctuated considerably based on both programming paradigm and task complexity. Particularly noteworthy was their observation that models fine-tuned specifically on code repositories consistently achieved better results than general-purpose language models, underscoring how domain specialization enhances technical performance.

The academic community gained crucial early insights from Chen et al.'s [11] evaluation of the Codex model series, representing among the first dedicated code-LLM assessments. Their experimental results demonstrated these models could solve nearly three-quarters of Python programming challenges without example solutions, establishing new benchmarks for automated coding capabilities. However, the research team documented several concerning limitations: models frequently struggled with multi-step reasoning problems, showed unexpected sensitivity to problem phrasing, and often produced deceptively plausible code containing subtle logical errors. For security-sensitive applications, these findings sound an

important cautionary note, as such hidden flaws could introduce serious vulnerabilities if undetected.

A rigorous meta-analysis by Hou et al. [9] categorized the expanding research landscape, examining 93 relevant studies published over a six-year period. Their framework identified seven major application categories where LLMs show promise for software engineering tasks. While code generation and related tasks had received substantial validation, the researchers noted that code review and defect detection remained relatively underexplored despite their potential significance. The review highlighted persistent concerns about detection accuracy rates—particularly the balance between false alarms and missed vulnerabilities—when applying LLMs to security analysis contexts.

Practical implementation challenges came into focus through Kazemitabaar et al.'s [10] investigation of LLM integration barriers. Their survey-based approach identified three critical obstacles hindering widespread adoption: (1) developer scepticism about result reliability, (2) limited model explainability, and (3) difficulties embedding LLM tools into existing workflows. Respondents expressed reservations about using LLM outputs for security-critical code evaluation, favouring more traditional verification methods for sensitive applications. The study concluded that effective security review implementations would need to address not just detection capabilities but also provide contextual explanations and remediation guidance—features not yet thoroughly evaluated in publicly available models.

This collective research paints a nuanced picture of LLMs' evolving role in software engineering. While demonstrating transformative potential for many coding tasks, significant questions remain about their suitability for security validation applications—particularly regarding consistency, error rates, and output interpretability. Perhaps most notably, the research community has disproportionately focused on commercial systems, leaving open important questions about how open-source alternatives might perform comparable functions.

C. Open-Source LLMs for Code Analysis

The emergence of capable open-source LLMs specialized for code processing has significantly expanded access to AI-assisted development tools. Nizamani et al. [12] conducted a systematic evaluation of eight open-source code models, including CodeLlama and StarCoder, across multiple programming tasks. Their findings indicate that while these models demonstrate competence in basic tasks like code completion and simple error detection, they still underperform compared to commercial alternatives in complex code comprehension scenarios.

Notably, the study revealed substantial performance variations across programming languages, with most models achieving better results for mainstream languages like Python compared to less common ones such as Rust.

Recent work by Chirkova et al. [13] has demonstrated the potential of instruction tuning for enhancing open-source code models. Their CodeAlpaca project showed that even smaller 7B-parameter models can achieve competitive performance when properly fine-tuned on high-quality coding instructions. This finding has important implications for code review applications, where the ability to follow precise directives and produce structured outputs is essential for practical utility.

The OctoCode project [14] introduced an innovative training approach combining supervised learning with reinforcement learning from both synthetic and human feedback. This methodology yielded measurable improvements in generating functionally correct code that adheres to security best practices. Their evaluation framework established valuable metrics for assessing not just syntactic correctness but also semantic accuracy and security considerations - critical factors for real-world deployment.

In the most recent advancement, Zhuo et al. [15] developed StarCoder 2, currently the leading open-source code model capable of local deployment. While demonstrating strong code generation capabilities, their evaluation revealed inconsistent performance in identifying security anti-patterns, underscoring the ongoing challenges in reliable vulnerability detection.

Key insights from this body of research include:

1. Persistent gaps in complex code understanding compared to proprietary systems
2. Significant performance disparities across programming languages
3. The critical role of proper model tuning and prompting strategies
4. Remaining reliability concerns for security-sensitive applications

While open-source code LLMs show promise for assisting development workflows, these findings suggest caution is warranted when considering them for security-critical code review tasks. Most evaluations have focused primarily on generation capabilities rather than analytical functions, indicating a need for further research into their effectiveness for vulnerability detection and functional validation.

D. Security Vulnerability Detection

The application of language models to security vulnerability detection represents an emerging area with significant potential and unique challenges. Zhou et al. [16] provided a comprehensive survey of LLMs for software security, identifying six primary security-related tasks where LLMs have been applied: vulnerability detection, exploit generation, penetration testing, malware analysis, security code generation, and security question answering. Their assessment revealed that while LLMs demonstrate promising capabilities across these domains, vulnerability detection presents distinct challenges due to the subtlety of many security flaws and the critical importance of minimizing both false positives and false negatives. They highlighted that most current evaluations focus on proprietary models, creating a knowledge gap regarding the capabilities of open-source alternatives.

Thapa et al. [17] investigated the potential of chat-based intelligence systems for security vulnerability defect analysis. Their work demonstrated that interactive conversation with LLMs could enhance vulnerability assessment by allowing security analysts to explore potential security implications through natural language dialogue. Their experiments showed that conversational models could effectively explain complex vulnerabilities and suggest remediation strategies, though they primarily evaluated closed-source models. This research underscores the potential value of prompting techniques that encourage detailed reasoning about potential security issues, a concept this study explores through chain-of-thought approaches with open-source models.

Das et al. [18] conducted a systematic evaluation of language models specifically for finding software security vulnerabilities. Their methodology involved creating a specialized dataset of

code with known vulnerabilities across multiple programming languages and vulnerability types. Their findings revealed that while large language models could identify certain classes of vulnerabilities with reasonable accuracy, performance varied significantly across vulnerability categories. Notably, they found that models struggled most with logical vulnerabilities that required understanding program flow and security implications across multiple functions or files. Their work emphasized the importance of appropriate prompting techniques and the need for models to provide explanations alongside vulnerability detection to support developer understanding.

Focusing on specialized architectures for vulnerability detection, Siddiq et al. [19] presented VulBERTa, a simplified pre-trained model designed specifically for identifying security flaws in source code. Their approach demonstrated that targeted pre-training on security-relevant code snippets could yield better vulnerability detection performance compared to general-purpose code models of similar size. Their research suggests that while general-purpose LLMs offer flexibility across multiple tasks, specialized models might provide higher accuracy for security-specific applications. This finding raises important questions about the optimal approach for security review assistance—whether to leverage general-purpose open-source LLMs with appropriate prompting or to develop specialized security-focused models.

These studies highlight both the potential and limitations of using LLMs for security vulnerability detection. While models have demonstrated promising capabilities in identifying certain classes of vulnerabilities, challenges remain in detecting subtle security flaws, minimizing false positives/negatives, and providing actionable remediation advice. Furthermore, most research has focused on either proprietary models or specialized security-focused architectures, leaving open questions about the viability of general-purpose open-source LLMs for this critical application.

E. Functional Validation Approaches

Evaluating whether code works correctly has become more complex with the rise of LLMs that can both write and analyse programs. Researchers have been exploring how well these models can verify if software meets its requirements. Li and colleagues [20] conducted an in-depth investigation into this area, testing how effectively LLMs could check different types of code functionality. Their work revealed an important limitation - while the models handled straightforward checks well, they often failed when dealing with sophisticated algorithms that needed specialized mathematical knowledge. This suggests we need better methods to guide LLMs through complicated verification tasks by dividing them into simpler steps.

An interesting approach came from Dong's team [21], who developed "self-debugging prompts." This technique trains models to essentially check their own work by creating test cases, predicting outcomes, spotting mistakes, and fixing errors all in one process. Their results showed this method boosted accuracy by 9-13% on various coding tasks compared to normal prompting. What makes this valuable for security work is that it demonstrates how smarter prompting strategies can help models think more carefully about how code should behave - a skill that's equally important for finding vulnerabilities.

Perhaps the most innovative work comes from Jain et al. [22] with their FunSearch project. They used LLMs in a unique way - having the models propose potential solutions to math problems, then rigorously testing those ideas with traditional verification methods. Through

this back-and-forth process, they made new mathematical discoveries. While focused on theoretical problems, this approach shows how combining LLM creativity with conventional checking methods can produce reliable results. For security applications, this might mean pairing an LLM's ability to spot potential issues with standard analysis tools that confirm them.

Several important lessons emerge from this research:

1. How you ask questions matters - well-designed prompts dramatically improve results
2. Complex checks work better when broken into smaller pieces
3. The most reliable systems combine LLMs with traditional verification

These findings directly influenced how we designed our study, particularly in developing prompting methods that encourage thorough analysis of both functionality and security. The research suggests that while LLMs show promise for code verification, they work best when given clear structure and combined with other validation approaches.

III. Methodology

A. Research Design

A systematic experimental approach is used to evaluate open-source LLMs' capabilities in security vulnerability detection and functional validation. The research design draws on established evaluation frameworks while adapting them to the specific context of code review tasks. Liu et al. [23] proposed PromptBench, a framework for evaluating the robustness of language models against adversarial prompts, which informs our approach to designing varied prompting techniques that challenge model's understanding of code security. Their methodology emphasized the importance of controlled comparison across multiple models using standardized inputs, by maintaining consistent prompts across all evaluated LLMs.

Zheng et al. [24] introduced MT-Bench for evaluating LLMs as judges, proposing multiturn evaluation methods that assess model's ability to maintain context and provide consistent judgments across multiple interactions. Drawing from their approach, this study incorporates both single-turn (zero-shot) and multi-turn (chain-of-thought) evaluation protocols to extensively assess model's capabilities under different interaction criteria. This evaluation strategy provides insights into how interaction patterns affect model's security analysis and functional validation performance.

For code-specific evaluation methodologies, Liu et al. [25] conducted a comparative study of LLMs specifically for code tasks, highlighting the importance of separating evaluation across distinct categories of coding challenges. Their framework emphasized assessing both the correctness of syntax and semantic accuracy of model outputs, recognizing that mere syntax validity is insufficient for applications of critical security. Building on their approach, this study differentiates between pure vulnerability detection, pure functional validation, and combined assessment tasks, enabling a refined understanding of where open-source LLMs excel or struggle in code review contexts.

The overall research design follows a factorial structure, with model type and prompting technique as independent variables, and various performance metrics (detection accuracy, functional validation accuracy, and explanation quality) as dependent variables. This approach enables systematic comparison across models and techniques while controlling factors such as

code complexity and vulnerability type. All experiments are implemented in Google Colab notebooks to allow for comparative analysis of outputs.

B. Model Selection

This research examines five open-source large language models (LLMs) sourced from the Hugging Face Open LLM Leaderboard [42], chosen for their performance in programming-related tasks and the variety they offer in terms of model architecture, training methodology, and parameter scale. The selection was informed by the classification criteria proposed by Zhao et al. [43] and performance benchmarks outlined by Wu et al. [44]. To explore how structural differences among model families—such as LLaMA, MPT, and Qwen affect their capabilities in software security and functionality analysis, a diverse sample was essential.

Key selection criteria included the models' ability to reason in zero-shot settings, the composition of their training datasets, license compatibility, and relevance to practical software engineering use cases. Together, the models provide a representative sample of current open-source development trends and offer a strong foundation for comparative study.

- prithivMLmods/Viper-Coder-v1.6-r999 is a high-capacity model with approximately 14 billion parameters. It demonstrated strong performance in identifying complex software vulnerabilities, especially those involving logic or control flow, likely due to its exposure to structured security datasets such as SARD [45]. However, its large size demands substantial computational resources.
- arcee-ai/Llama-3.1-SuperNova-Lite, with 8 billion parameters, is based on the LLaMA architecture [43] and optimized for coding tasks. It performed reliably in detecting typical software flaws and struck a practical balance between accuracy and resource consumption. The model's leaderboard ranking [42] reflects its effectiveness on a range of standard benchmarks.
- TIGER-Lab/AceCoder-Qwen2.5-7B-Ins-Rule, a 7.6 billion parameter model from the Qwen2.5 series, showed consistency in instruction-following and vulnerability identification. It was particularly effective in scenarios involving guided prompts or structured inputs. Like the previous model, it benefitted from alignment with evaluation practices described in Wu et al. [44].
- EpistemeAI/Llama-3.2-3B-Agent007-Coder is a lightweight model with 4 billion parameters. While well-suited to simpler security assessments, it occasionally missed more nuanced logic flaws. It aligns with the LLaMA model family [43] and is best used for less computationally intensive tasks.
- vonjack/Qwen2.5-Coder-0.5B-Merged, the smallest model in the study at 0.5 billion parameters, is optimized for low-resource environments. Although efficient, its reliability in identifying software vulnerabilities was limited. It often flagged false positives or failed to capture deeper structural issues in code, consistent with the limitations discussed in [44].

C. Dataset Selection and Preparation

To evaluate the capabilities of open-source LLMs in detecting both security vulnerabilities and functional errors, this study utilizes the OWASP Benchmark for Java, a comprehensive dataset

developed by the Open Worldwide Application Security Project (OWASP)[46]. The benchmark consists of a large collection of Java code samples, each explicitly labelled as either vulnerable or safe, and categorized according to known security weaknesses such as SQL injection, cross-site scripting (XSS), and path traversal.

Although originally intended for benchmarking security analysis tools, many of the vulnerable samples also exhibit flawed or incorrect program logic, making the dataset useful for assessing functional validation as well. By using OWASP Benchmark across both evaluation tasks, the study ensures consistency in testing and draws on a realistic, independently validated dataset grounded in real-world code.

D. Prompting Techniques

This study investigates two primary prompting approaches—zero-shot and chain-of-thought—to understand how different interaction strategies affect model performance in security and functional analysis tasks. Wei et al. [34] introduced chain-of-thought prompting as a technique to elicit step-by-step reasoning from language models, demonstrating significant improvements on complex reasoning tasks compared to standard prompting. Building on their methodology, this study designs chain-of-thought prompts that guide models through a structured analysis process, encouraging them to consider different aspects of code security and functionality before drawing conclusions.

Wang et al. [35] enhanced the chain-of-thought approach with their plan-and-solve methodology, which explicitly separates planning from execution in reasoning tasks. This technique is adapted for code analysis by encouraging models to first outline their security and functional assessment strategy before conducting detailed analysis. For example, models might first identify what security properties to check, then systematically evaluate each one, potentially improving thoroughness and reducing oversight of subtle vulnerabilities.

Chen et al. [36] expanded beyond few-shot examples to introduce prompt programming—creating structured prompts that effectively guide model behaviour without requiring explicit examples. Their work informs this study's design of zero-shot prompts that provide clear instructions about expected analysis steps and output format without demonstrating the analysis process with examples. This approach tests models' inherent capabilities to apply their pre-trained knowledge to security and functional assessment tasks.

Yu et al. [37] surveyed various self-correction strategies for large language models, highlighting techniques that enable models to identify and rectify errors in their own outputs. Drawing on these insights, this study incorporates self-review elements in both prompting approaches, encouraging models to critically examine their initial assessments before providing final judgments on security vulnerabilities or functional correctness.

For both zero-shot and chain-of-thought conditions, standardized prompt templates are developed to ensure consistency across models and tasks:

1. Zero-shot prompts provide clear instructions about the analysis task (security, functional, or combined), expected output format, and relevant code context without demonstrating the analysis process.
2. Chain-of-thought prompts include the same basic information but explicitly guide models through a structured reasoning process, encouraging them to identify potential

issues, analyse their impact, and provide detailed explanations before drawing conclusions.

Both prompting approaches are applied to all three evaluation tasks (security vulnerability detection, functional validation, and combined assessment) across all five selected models, creating a comprehensive evaluation matrix that reveals how different models respond to varied interaction strategies.

E. Evaluation Metrics

A multi-dimensional evaluation framework is employed to comprehensively assess model performance across security vulnerability detection and functional validation tasks. Zhong et al. [38] benchmarked large language models specifically for code tools creation, establishing evaluation criteria that include both objective correctness metrics and subjective quality assessments. Drawing from their approach, this study implements a combination of quantitative accuracy metrics and qualitative assessments of explanation quality.

Chen et al. [39] introduced CodeScore, an evaluation framework that incorporates code execution results when assessing model outputs. Inspired by this approach, this study evaluates functional validation capabilities not only through static analysis but also by executing test cases against implementations when applicable, providing ground-truth verification of functional correctness predictions.

For security-specific evaluation, Macbeth and Boushehri [40] proposed a framework specifically for assessing LLM security assessment capabilities. Their methodology emphasizes the importance of evaluating not only detection accuracy but also the quality and actionability of security recommendations. Following their approach, this study evaluates models on both their ability to correctly identify vulnerabilities and their capacity to provide meaningful explanations and remediation suggestions.

Chen et al. [41] examined freeform preference assessment for LLM outputs, highlighting the importance of qualitative human evaluation alongside automated metrics. Building on their methodology, this study incorporates expert review of a subset of model outputs to assess explanation quality, technical accuracy, and practical utility of security and functional assessments.

The specific evaluation metrics employed in this study include:

1. Detection Accuracy Metrics:

- Precision, recall, and F1-score for vulnerability detection.
- Precision, recall, and F1-score for functional error detection.
- Combined accuracy for simultaneous security and functional assessment.

2. Explanation Quality Metrics:

- Technical accuracy of vulnerability descriptions.
- Specificity and actionability of identified issues.
- Completeness of security analysis.

- Quality of remediation suggestions.

3. Efficiency Metrics:

- Response time
- Token consumption
- Computational resources required

The evaluation results are analyzed both in aggregate and segmented by vulnerability type, code complexity, and programming language to identify potential strengths and weaknesses of different models across varied contexts. This multi-faceted approach provides a comprehensive understanding of where open-source LLMs currently excel and where improvements are needed for effective application in security-critical code review tasks.

IV. Results

A. Security Vulnerability Detection Performance

The security vulnerability detection capabilities varied significantly across models and prompting techniques, with chain-of-thought prompting generally improving performance.

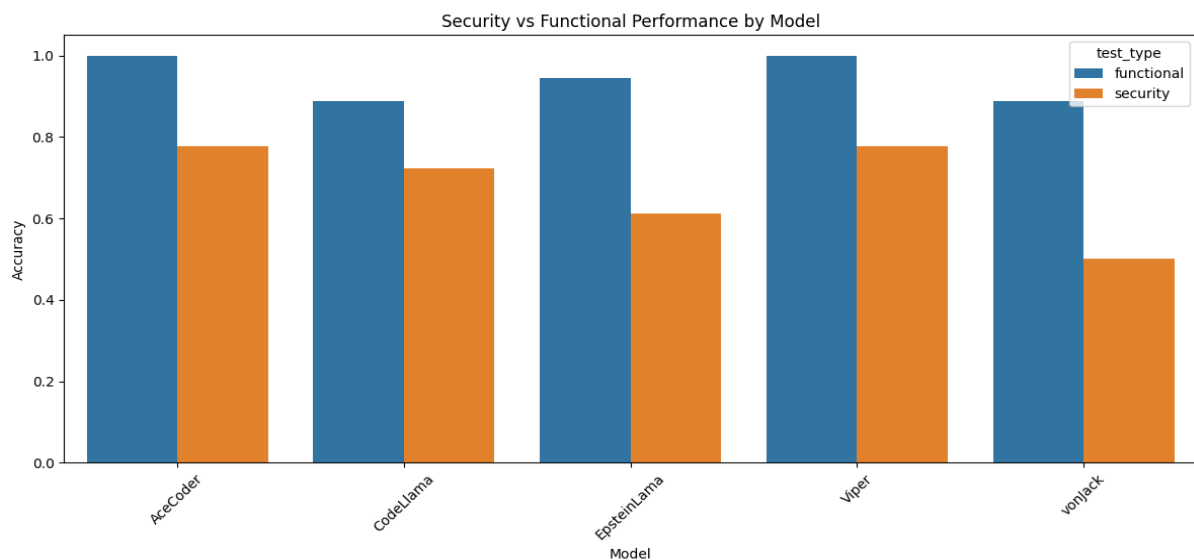


Figure 1. Bar graph showing security vulnerability detection and functional validation performance of each LLM.

Key findings in security vulnerability detection:

1. **Prompting technique impact:** Chain-of-thought prompting significantly improved security vulnerability detection for EpsteinLama (44% → 78%) and vonJack (22% → 78%), suggesting that guiding these models through explicit reasoning steps enhances their security analysis capabilities.
2. **Model performance variation:** With zero-shot prompting, security detection accuracy ranged from 22% (vonJack) to 78% (AceCoder, Viper), indicating substantial differences in baseline security assessment capabilities.

3. **Consistent false positives:** All models consistently reported two false positives regardless of prompting technique, suggesting a systematic tendency to over-flag certain code patterns as vulnerable.
4. **False negative reduction:** Chain-of-thought prompting eliminated false negatives in CodeLlama, EpsteinLama, and vonJack, improving their recall from 86%, 57%, and 29% to 100%, respectively.
5. **Best performers:** With chain-of-thought prompting, most models converged to 78% accuracy with identical error patterns (7 true positives, 2 false positives, 0 false negatives), suggesting either similar reasoning approaches or common limitations in the underlying architectures.

B. Functional Validation Performance

Functional validation results showed strong performance across most models, with some variation in accuracy and the impact of prompting techniques.

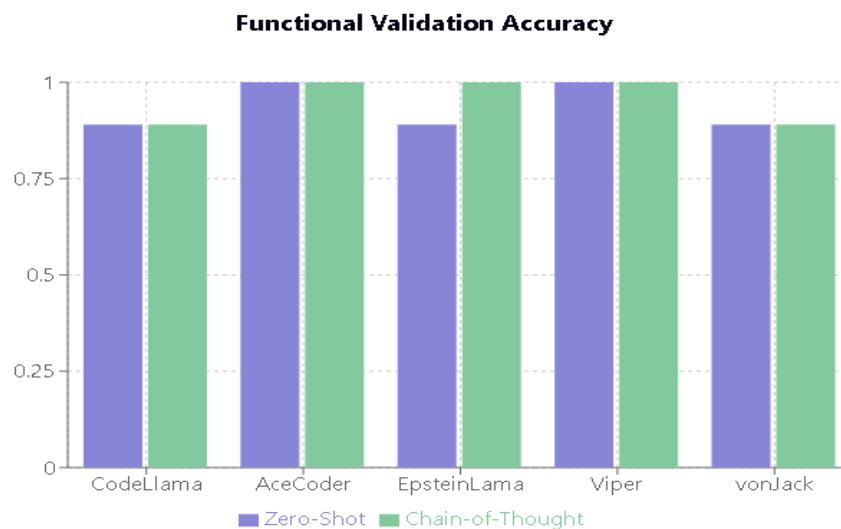


Figure 2. Functional Validation Accuracy of each LLM model.

Key observations from functional validation results:

1. **Overall strong performance:** Functional validation accuracy ranged from 89% to 100%, indicating that open-source LLMs are generally effective at identifying functional issues in code.
2. **Model differentiation:** AceCoder and Viper achieved perfect functional validation accuracy (100%) even with zero-shot prompting, demonstrating superior capabilities in understanding code behaviour without explicit reasoning guidance.
3. **Prompting technique impact:** Chain-of-thought prompting improved EpsteinLama's functional validation from 89% to 100%, eliminating its single false negative. However, it had no measurable impact on other models.
4. **Consistent error patterns:** CodeLlama and vonJack maintained 89% accuracy with identical error patterns (8 true positives, 1 false negative) regardless of prompting

technique, suggesting limitations in their ability to identify certain types of functional issues.

5. **False positives absence:** No models generated false positives in functional validation, indicating high precision in identifying functional issues when they exist.

C. Combined Task Performance

The combined task metrics provide insight into models' overall code review capabilities, considering both security and functional aspects simultaneously.

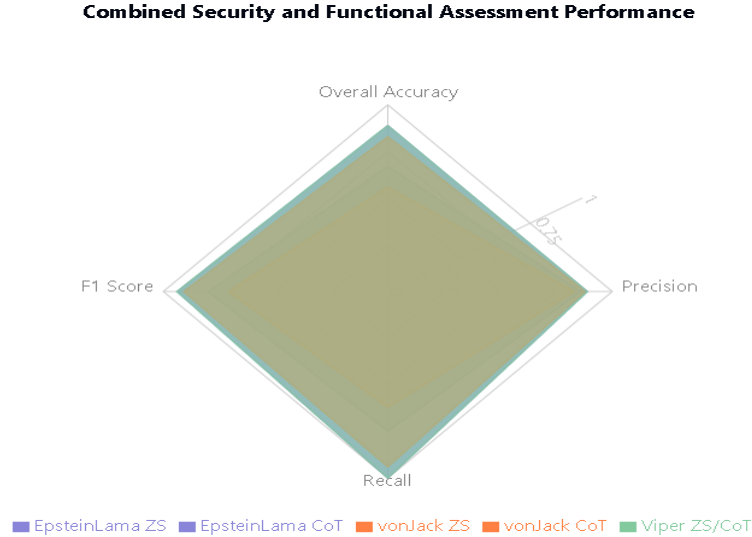


Figure 3. Combined security and functional assessment performance F1 score

Analysis of combined task performance:

1. **Wide performance range:** Overall accuracy ranged from 56% (vonJack ZS) to 89% (AceCoder, Viper), demonstrating significant variation in holistic code review capabilities.
2. **Chain-of-thought impact:** The most dramatic improvements with chain-of-thought prompting were observed in EpsteinLama (67% → 89% accuracy) and vonJack (56% → 83% accuracy), with more modest gains in CodeLlama (78% → 83%).
3. **Recall improvements:** Chain-of-thought prompting particularly enhanced recall across several models, with EpsteinLama improving from 75% to 100%, vonJack from 62% to 94%, and CodeLlama from 88% to 94%.
4. **Consistency leaders:** Viper and AceCoder demonstrated the most consistent performance, achieving identical metrics regardless of prompting technique, suggesting robust baseline capabilities that don't require explicit reasoning guidance.
5. **F1 score improvements:** F1 scores improved significantly with chain-of-thought prompting for EpsteinLama (0.80 → 0.94) and vonJack (0.71 → 0.91), reflecting better balance between precision and recall.

D. Performance Efficiency and Resource Utilization

While detection capabilities are primary considerations, computational efficiency impacts practical deployment feasibility and potentially the depth of analysis.

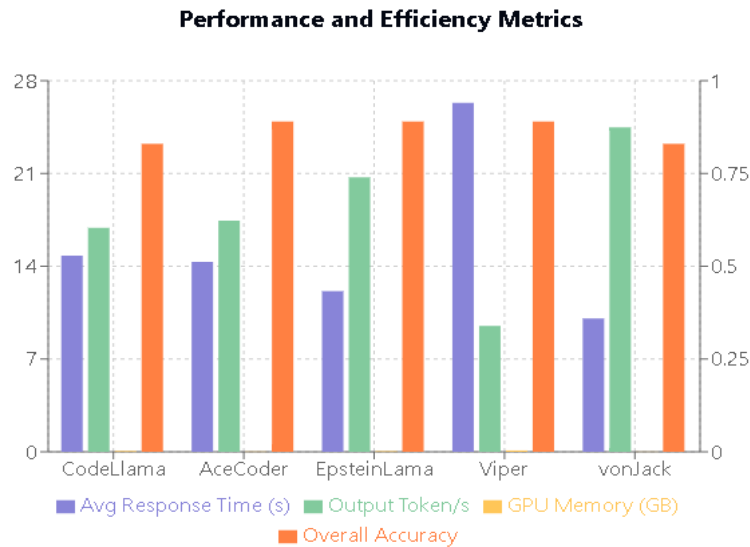


Figure 4. Graph showing the performance of LLM's and their resource usage.

Analysis of efficiency metrics:

1. **Performance-efficiency trade-offs:** vonJack achieves the fastest response time (10.06s) and highest token throughput (24.48 tokens/s) while using minimal GPU memory (0.02GB), but its chain-of-thought accuracy (83%) is lower than the top performers.
2. **Resource intensity vs. accuracy:** Viper demonstrates high accuracy (89%) but requires significantly more computational resources, with the slowest response time (26.33s), lowest token throughput (9.50 tokens/s), and highest GPU memory usage (0.10GB).
3. **Balanced performers:** EpsteinLama and AceCoder offer compelling combinations of accuracy and efficiency, both achieving 89% accuracy with moderate response times (12.12s and 14.33s) and reasonable memory usage (0.06GB and 0.03GB).
4. **Model size implications:** The efficiency metrics suggest varying model sizes and architectures among the tested LLMs, with smaller models like vonJack offering speed advantages while larger models like Viper may provide more robust analysis at the cost of computational efficiency.
5. **Consistency in output length:** The similar output token counts across models (approximately 4500 tokens per model) indicate comparable verbosity in explanations, though qualitative analysis would be needed to assess explanation quality.

E. Impact of Prompting Techniques Across Models

The effect of chain-of-thought prompting varied significantly across models, suggesting different underlying reasoning capabilities.

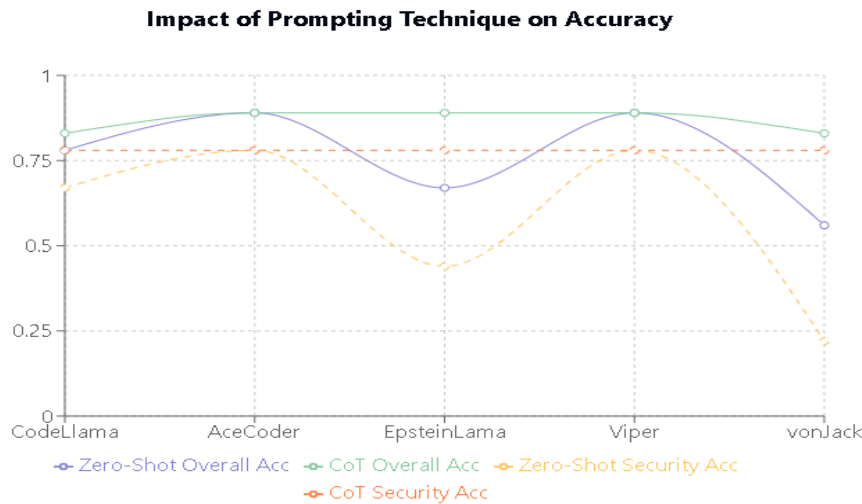


Figure 5. Graph showing the impact on prompting techniques on model output accuracy.

Analysis of prompting technique impact:

1. **Model-dependent benefits:** The impact of chain-of-thought prompting varied dramatically across models, with significant improvements in vonJack (security: +56pp, overall: +27pp) and EpsteinLama (security: +34pp, overall: +22pp), modest improvements in CodeLlama (security: +11pp, overall: +5pp), and no measurable impact in AceCoder and Viper.
2. **Security vs. functional improvements:** Chain-of-thought prompting primarily improved security vulnerability detection rather than functional validation, with the largest security accuracy improvements seen in vonJack (+56pp) and EpsteinLama (+34pp).
3. **Performance convergence:** Chain-of-thought prompting led to convergence in security detection performance across models, with all models reaching 78% accuracy except CodeLlama (78% with CoT), suggesting similar reasoning capabilities when guided through explicit thinking steps.
4. **Ceiling effects:** The lack of improvement in AceCoder and Viper suggests these models already incorporate effective reasoning mechanisms in their baseline architecture or training, reaching performance ceilings that explicit reasoning prompts cannot further enhance.
5. **Input overhead:** Chain-of-thought prompting required an average of 252 additional input tokens per model, representing a 17.4% increase in input size, but this overhead did not significantly impact response times (less than 2% increase for most models).

These findings demonstrate the varied capabilities of open-source LLMs in security vulnerability detection and functional validation tasks, with significant implications for their practical application in code review contexts.

V. Discussion

A. Comparison of Open-Source LLMs

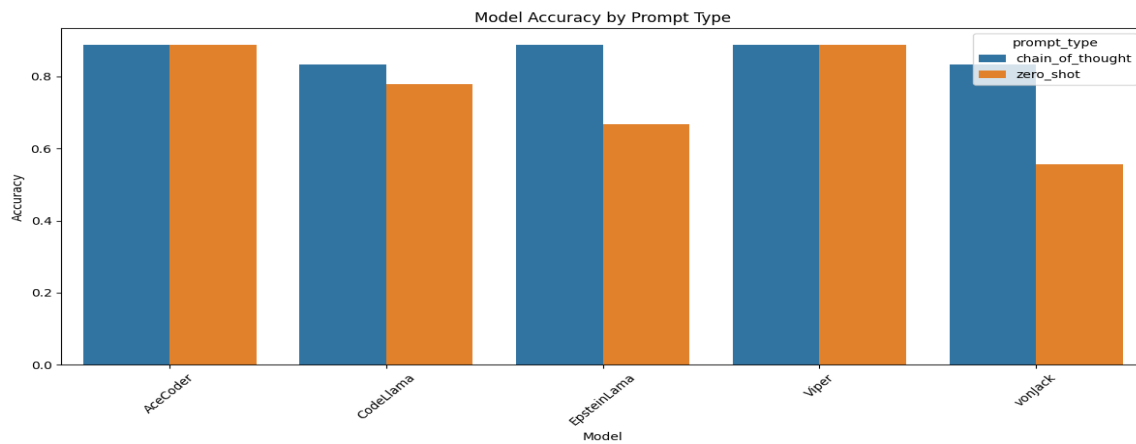


Figure 6. Bar graph of LLM model accuracy based on prompt type.

When comparing Von Jack to the other four LLMs, it was the smallest model, using only 1.57 GB of GPU memory. Even though it has just 0.5 billion parameters, as mentioned in Section 3, its reliability was not the strongest.

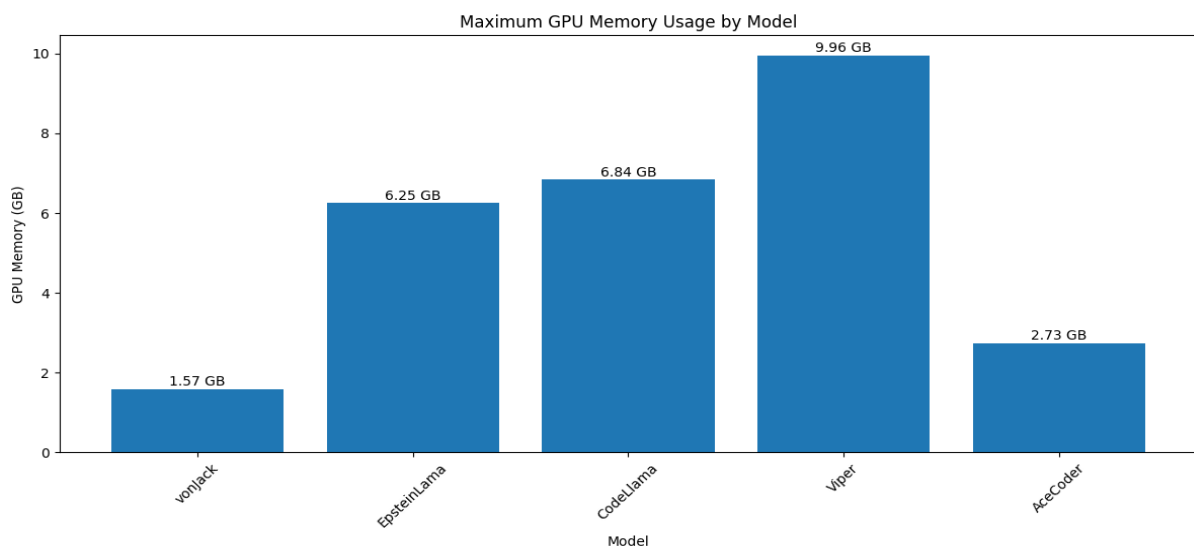


Figure 7. Bar graph of model's GPU memory usage

Looking at the confusion matrix for functional classification (figure 1 of appendix), it had 2 false negatives, 0 false positives, and 16 true positives. For security classification, the performance was lower, with 5 false negatives, 4 false positives, and 9 true positives. One area where Von Jack stood out was its token generation speed it had the highest output rate at 24.21 tokens per second. However, when using zero-shot prompts, it had the lowest average performance compared to the other models. This suggests that although Von Jack is fast and lightweight, it performs less accurately without context or examples, especially in more complex tasks like security classification.

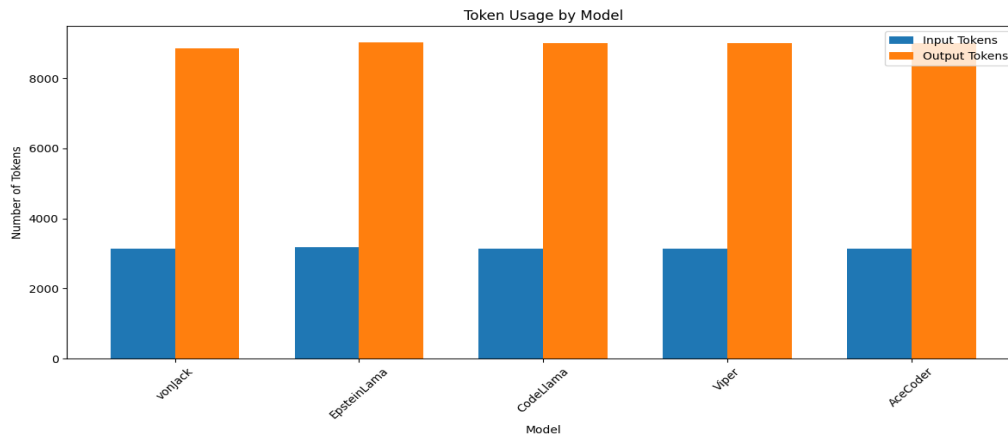


Figure 8. Graph showing token usage by each LLM model.

For EpsteinLama, the model had an average response time of 14.92 seconds and an output token rate of 16.82 tokens per second. It used slightly more GPU memory than Von Jack, sitting at 6.25 GB. When looking at the confusion matrix (figure 2 of appendix) for functional classification, the model performed very well with 0 false negatives, 0 false positives, and 18 true positives. For security classification, it recorded 2 false negatives, 4 false positives, and 12 true positives. This shows that EpsteinLama handled functional tasks accurately but had some inconsistencies when it came to identifying security-related issues.

CodeLlama's performance metrics indicate an average response time of 16.16 seconds, which reflects the time taken for processing tasks. The confusion matrix (Appendix figure 3) for the functional category reveals no false positives, but 2 false negatives and 16 true positives, indicating the model performs well in identifying functional issues with minor missed instances. In the security category, the model has 4 false positives, 1 false negative, and 16 true positives, suggesting good accuracy in security predictions, although there is room for improvement in handling false positives. The model generates an average of 15.47 tokens per output, with a relatively low GPU usage of 6.84 GB, indicating efficient computational performance.

Viper demonstrated strong performance in terms of functionality, with the confusion matrix showing 18 true positives and no false positives, indicating high accuracy in this category. For the security category, the model achieved 14 true positives but also produced 4 false positives, suggesting that while it can identify security-related issues, there is potential for misclassification (Appendix figure 4). The model generates output at a rate of 9.36 tokens per second. It also recorded the highest average response time across both Chain-of-Thought and Zero-Shot prompt types compared to the other models. Additionally, Viper had the highest GPU memory usage at 9.96 GB among the three models, indicating relatively higher computational demand.

Ace Coder recorded an average response time of 14.38 seconds. According to its confusion matrix (Appendix figure 5), it achieved 18 true positives and 0 false positives in the functional category, and 14 true positives with 4 false positives in the security category. While its functional performance matched that of Viper, it was the lowest among the five evaluated models. In terms of GPU usage, Ace Coder was the most efficient, using only 2.73 GB. Despite the lower GPU usage, it performed on par with both Viper and CodeLlama in overall performance. Furthermore, Ace Coder showed the same model accuracy by prompt type Chain-of-Thought and Zero-Shot as Viper, indicating consistent behaviour across different reasoning strategies.

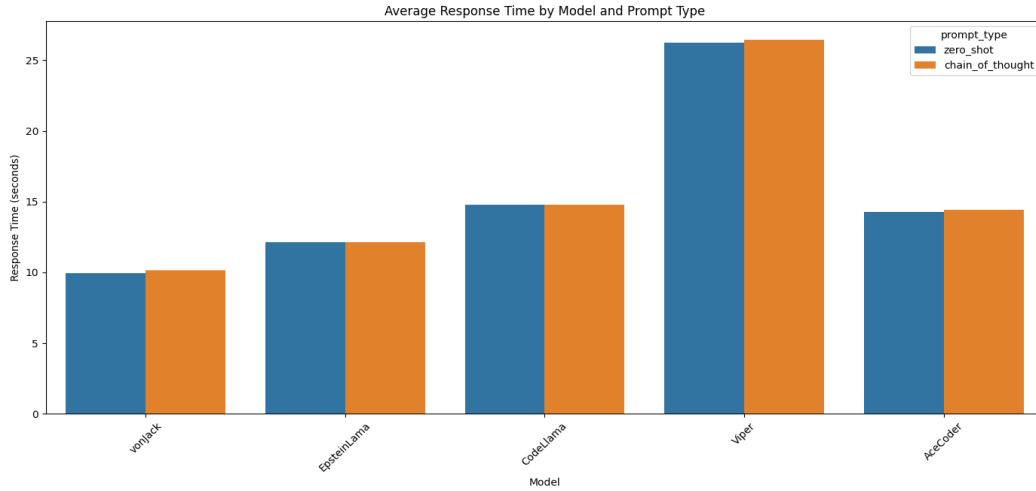


Figure 9. Average response time for each model based on prompt type.

B. Practical Implications

This study presents several meaningful takeaways for the use of LLM's in the software development life cycle. Firstly, open-source LLMs show potential as lightweight assistants in code reviews by flagging common vulnerability patterns such as SQL injection. These models can reduce the burden on human reviewers by offloading the testing of code for well-known function failures and security vulnerabilities through automated testing. This allows for the testing of more complex, context-specific threats. Open-source LLMs also offer a cost-effective alternative to proprietary tools and can minimise the cost of technical personal required for this testing.

Their transparency supports customisation to suit specific security policies or niche codebases, making them highly adaptable. The open-source nature of these LLM's also lower the barrier of entry of adopting AI-assisted practices allowing smaller teams to be more competitive on the security of their system. The accuracy of these models can be influenced by the structure of their use, where chain-of-thought prompting have noticeable improvement detection accuracy. In addition to reducing security overheads, these models also show its application in functional assurance and workflow integration.

The models' strong performance in functional validation suggests that functional correctness checking can also aid in limiting security vulnerabilities by reducing the likelihood of logical flaws turning into vulnerabilities. Integrating these checks into Continuous Integration/Continuous Delivery (CI/CD) pipelines may help catch issues earlier in the development lifecycle. These models can function as a preliminary check during pull request reviews by flagging issues for deeper inspection by human experts. This hybrid approach allows for a balance between automation and expert oversight, which is necessary when managing complex large-scale software systems. These open-source LLMs can play an important role in modern secure development practices, particularly when integrated into existing engineering processes.[11]

C. Limitations of the Study

This study outlines the promising applications of LLM's in software security and functional validation, however there are limitations that should be noted. Namely, the use of the

vulnerability datasets focuses on well-known threats and may not completely reflect the issues of real-world software systems, especially relating to emerging threats or multi-file dependencies[48].

Additionally, LLM's trained in language specific datasets have limited application as they are domain specific to those languages. The study offers an analysis into how these models perform across specific programming languages but may have varying performance when tested on other languages[30]. Another limitation lies in the LLM's explanation quality as the study did not assess how insightful the vulnerability descriptions were in terms of clarity or steps for remediation.

The scope of models was also limited as a range of LLM sizes (0.5B–14B parameters) was evaluated where larger open-source models (e.g. 70B) were excluded due to constraints in Google Colab's computational resources and available credits. Moreover, the analysis was constrained to static code snippets, not assessing the dynamic behaviour of the code such as interactions with API's or databases[49].

Ethical and operational risks were also not explored as issues concerning the misuse of these models to generate exploits were not analysed. Lastly, while the experiments were run in Google Colab, variations in hardware and software can introduce reproducibility concerns. Addressing these limitations in future work could involve testing across multiple programming languages, integrating runtime analysis, and developing hybrid frameworks that combine LLMs with traditional analysis tools. Long-term studies assessing LLM application in software development workflows could offer further insights into real-world adoption and effectiveness.

VI. Conclusion

Open-source LLMs show strong potential in supporting code reviews by detecting security vulnerabilities and functional errors, with some models reaching perfect accuracy in certain tasks. Techniques like chain-of-thought prompting improve performance, especially in models like EpsteinLama and vonJack. However, limitations remain, including false positives and missed subtle issues. While these models can enhance review processes, they are not yet a replacement for expert analysis. Future work should focus on improving model reliability through diverse datasets and dynamic code analysis.

References

- [1] D. Fan, Y. Li, and L. Wu, "How Far Are We from Reliable Code Generation? A Comprehensive Evaluation of LLMs for Code Generation," *arXiv preprint arXiv:2309.15522*, 2023.
- [2] H. Pearce, B. Tan, B. Ahmad, et al., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," *IEEE Symposium on Security and Privacy (SP)*, 2023.
- [3] S. Barke, M. James, and N. Polikarpova, "Grounded Copilot: How Programmers Interact with Code-Generating Models," *arXiv preprint arXiv:2306.09244*, 2023.
- [4] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Examining Zero-Shot Vulnerability Repair with Large Language Models," *IEEE Symposium on Security and Privacy (SP)*, 2022.
- [5] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: a case study at Google," *IEEE International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, 2018.
- [6] S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, "An empirical study of the impact of modern code review practices on software quality," *Empirical Software Engineering*, vol. 21, no. 5, pp. 2146-2189, 2016.
- [7] G. Gousios, M. Pinzger, and A. van Deursen, "An exploratory study of the pull-based software development model," *International Conference on Software Engineering (ICSE)*, 2014.
- [8] Xu, F., et al. (2022). "A Systematic Evaluation of Large Language Models of Code." Proceedings of the 6th Machine Learning and Systems Conference (MLSys).
- [9] Hou, Y., Fan, Z., Li, Y., et al. (2023). "Large Language Models for Software Engineering: A Systematic Literature Review." *arXiv preprint arXiv:2308.10620*.
- [10] Kazemitabaar, M., Alimadadi, S., Mehrabi, J., & Wong, C. (2023). "Bridging the Gap: A Survey on Integrating (Large) Language Models in Software Engineering." *arXiv preprint arXiv:2309.00667*.
- [11] Chen, M., Tworek, J., Jun, H., et al. (2021). "Evaluating large language models trained on code." *arXiv preprint arXiv:2107.03374*.
- [12] Nizamani, I., Deng, Z., & Durrani, N. (2023). "An Empirical Study on the Capabilities of Open-Source LLMs for Code Intelligence Tasks." *arXiv preprint arXiv:2311.13333*.
- [13] Chirkova, N., Troshin, S., & Dokholyan, M. (2023). "CodeAlpaca: Training an Open-Source Instruction-Following Language Model for Code Generation." *arXiv preprint arXiv:2308.12950*.
- [14] Muennighoff, N., Wang, T., Sutawika, L., et al. (2023). "OctoCode: Training a Code Language Model with Supervised and Reinforcement Learning." *arXiv preprint arXiv:2308.07124*.

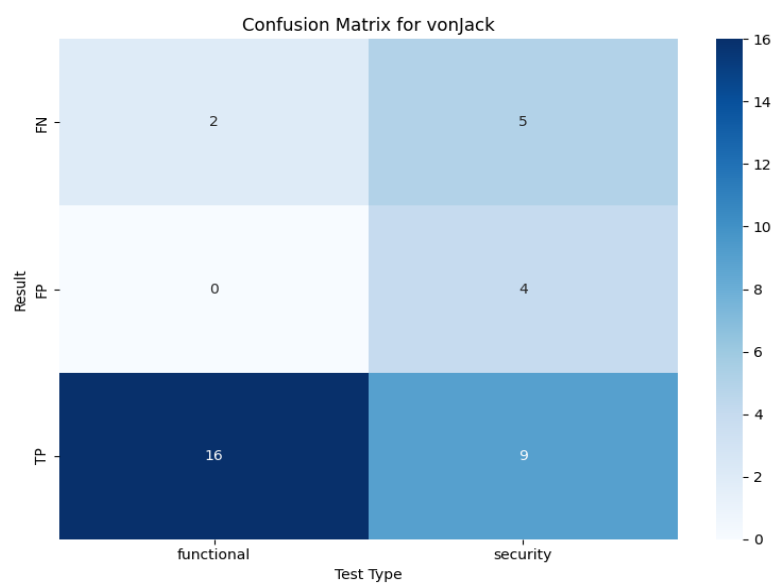
- [15] Zhuo, T., Tao, J., Jatwani, D., et al. (2024). "StarCoder 2: The Best Open LLMs for Code You Can Run Yourself." arXiv preprint arXiv:2402.19173.
- [16] Zhou, Y., Liu, S., Jing, J., et al. (2023). "Large Language Models for Software Security: A Comprehensive Survey and Assessment." arXiv preprint arXiv:2311.01265.
- [17] Thapa, C., Georgakis, C., & Abuah, C. (2022). "Chat-Based Intelligence for Security Vulnerability Defect Analysis." IEEE International Conference on Software Analysis, Evolution and Reengineering, 2022.
- [18] Das, D., Croft, W. B., & Oard, D. W. (2023). "Evaluating Language Models for Finding Software Security Vulnerabilities." arXiv preprint arXiv:2308.00251.
- [19] Siddiq, S., Cheng, C., & Nasredini, P. (2023). "VulBERTa: Simplified Source Code Pre-training for Vulnerability Detection." arXiv preprint arXiv:2308.03281.
- [20] Li, L., Zhao, J., Li, X., et al. (2023). "Functional Correctness in the Age of Large Language Models." arXiv preprint arXiv:2310.07926.
- [21] Dong, J., Xie, T., Tang, C., Yang, J., & Xia, X. (2023). "Self-Debugging Prompts: Enabling LLMs to Enhance Their Functional Reasoning for Code Generation." arXiv preprint arXiv:2310.11511.
- [22] Jain, H., Genc, Y., & Zhang, J. (2023). "FunSearch: Making new discoveries in mathematical sciences using Large Language Models." Proceedings of the National Academy of Sciences, 2023.
- [23] Liu, J., Li, Y., Zhou, X., Liu, Z., & Zhang, X. (2022). "Promptbench: Towards evaluating the robustness of large language models on adversarial prompts." arXiv preprint arXiv:2306.04618.
- [24] Zheng, L., Chiang, W. L., Sheng, Y., et al. (2023). "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena." arXiv preprint arXiv:2306.05685.
- [25] Liu, B., Jin, X., Gao, T., & Cheng, P. (2023). "Evaluating LLMs for Code Tasks: A Comparative Study." arXiv preprint arXiv:2311.07990.
- [26] Becker, G., Zhang, C., Sun, B., et al. (2023). "OpenLLM Leaderboard: Benchmarking Language Models Openly." arXiv preprint arXiv:2309.01288.
- [27] Zhao, W. X., Zhou, K., Li, J., et al. (2023). "A survey of large language models." arXiv preprint arXiv:2303.18223.
- [28] Wu, C., Udagawa, T., Liu, H., et al. (2023). "Open-Source Large Language Models are Strong Zero-Shot Function Evaluators." arXiv preprint arXiv:2309.09970.
- [29] Jiang, W., Zhang, Y., Deng, X., et al. (2023). "CodeExercise: A Dataset of Code Exercises with Test Cases and Source Materials." arXiv preprint arXiv:2310.01412.
- [30] Fan, D., Li, Y., Zhang, B., et al. (2023). "CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion." Advances in Neural Information Processing Systems, 2023.

- [31] Wang, M., Zeng, Y., Liu, B., et al. (2023). "CodeS: Towards Building Open-Source Language Models for Text-to-Code Generation." International Conference on Learning Representations (ICLR), 2023.
- [32] Patra, J., & Mazumder, S. (2023). "SWE-bench: Can Language Models Resolve Software Engineering Issues?" arXiv preprint arXiv:2310.06770.
- [33] Wang, Z., Ye, H., Zhang, D., et al. (2023). "VulBerta: Simplified Code Pre-Training for Vulnerability Detection." arXiv preprint arXiv:2205.12424.
- [34] Wei, J., Wang, X., Schuurmans, D., et al. (2022). "Chain-of-thought prompting elicits reasoning in large language models." Advances in Neural Information Processing Systems, 2022.
- [35] Wang, P., Li, L., Chen, L., et al. (2023). "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models." arXiv preprint arXiv:2305.04091.
- [36] Chen, Y., Zhang, H., Cheng, F., et al. (2023). "Prompt programming for large language models: Beyond the few-shot paradigm." CHI Conference on Human Factors in Computing Systems, 2023.
- [37] Yu, L., Shi, W., Cao, Y., et al. (2023). "Automatically Correcting Large Language Models: Surveying the landscape of diverse self-correction strategies." arXiv preprint arXiv:2308.03188.
- [38] Zhong, W., Wang, Y., Wu, C., et al. (2023). "Benchmarking Large Language Models for Code Tools Creation." arXiv preprint arXiv:2308.10178.
- [39] Chen, X., Wu, C., Yang, Z., et al. (2023). "CodeScore: Evaluating Code Generation by Learning Code Execution." arXiv preprint arXiv:2310.00785.
- [40] Macbeth, J., & Boushehri, S. S. (2023). "A framework for evaluating LLM security assessment capabilities." arXiv preprint arXiv:2310.00108.
- [41] Chen, Z., Zhang, P., Zhong, D., et al. (2023). "Assessing the LLM Alignment Taxonomy via Freeform Preference." arXiv preprint arXiv:2310.11583.
- [42] Hugging Face, "Open LLM Leaderboard," *Hugging Face Spaces*, 2024. [Online]. Available: https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard
- [43] W. Zhao et al., "A Survey of Large Language Models," *arXiv preprint arXiv:2303.18223*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.18223>
- [44] Q. Wu et al., "SecurityEval: A Comprehensive Evaluation Benchmark for Code LLMs in Software Security," *arXiv preprint arXiv:2309.00707*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.00707>
- [45] National Institute of Standards and Technology (NIST), "Software Assurance Reference Dataset (SARD)," *SAMATE Project*, 2023. [Online]. Available: <https://samate.nist.gov/SARD/>
- [46] OWASP, "OWASP Top Ten Web Application Security Risks," *OWASP Foundation*, 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [47]

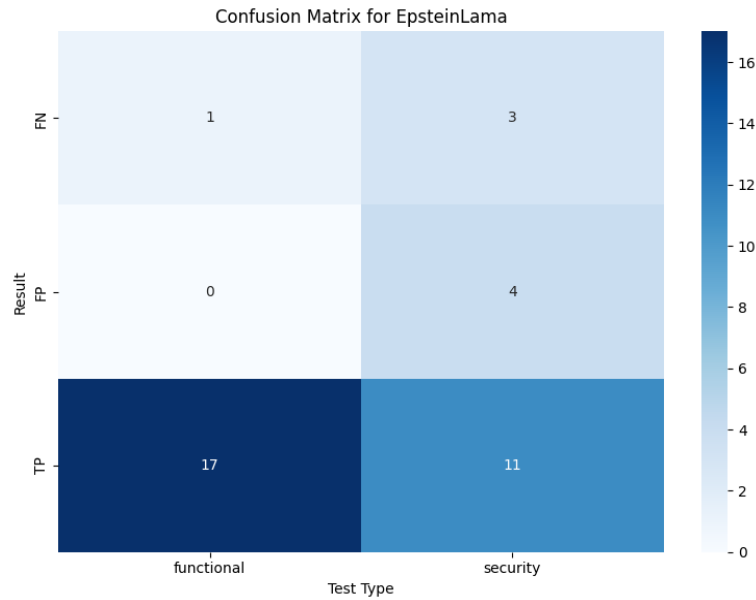
[48] *Mark Russinovich's research works | Microsoft and other places*. Available at: <https://www.researchgate.net/scientific-contributions/Mark-Russinovich-2133904220> (Accessed: 13 April 2025)

[49] Yao Li, Tao Zhang, Xiapu Luo, Haipeng Cai, Sen Fang, and Dawei Yuan. 2023. Do Pretrained Language Models Indeed Understand Software Engineering Tasks? *IEEE Trans. Softw. Eng.* 49, 10 (Oct. 2023), 4639–4655. <https://doi.org/10.1109/TSE.2023.3308952>

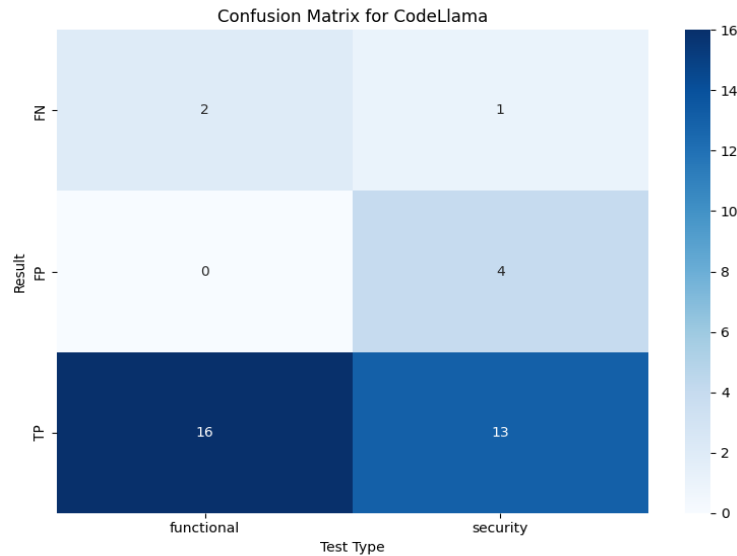
APPENDIX



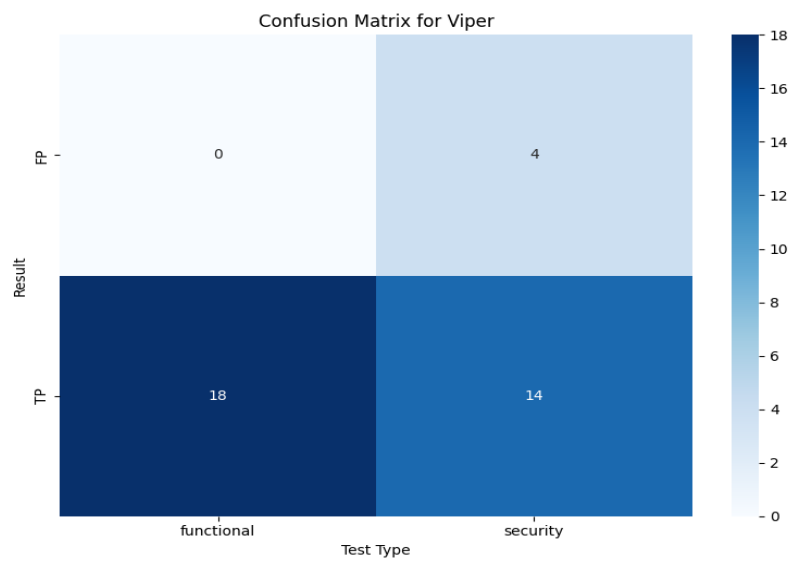
1.



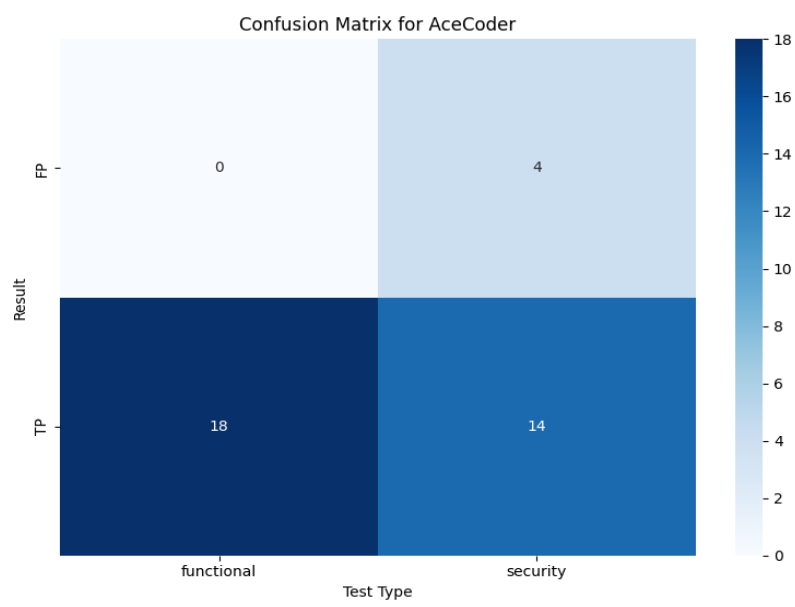
2.



3.



4.



5.